JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

# Introduction to Generative AI

## Retrieval-Augmented Generation

# Outline

- Module Learning Objectives
- Introduction
- Foundations of RAG
- Retrieval Techniques and Metrics
- Implementation and Probabilistic Methods
- Retrieval-Augmented LLMs (RA-LLMs)
- Training Strategies for RA-LLMs
- Challenges and Outlook
- References

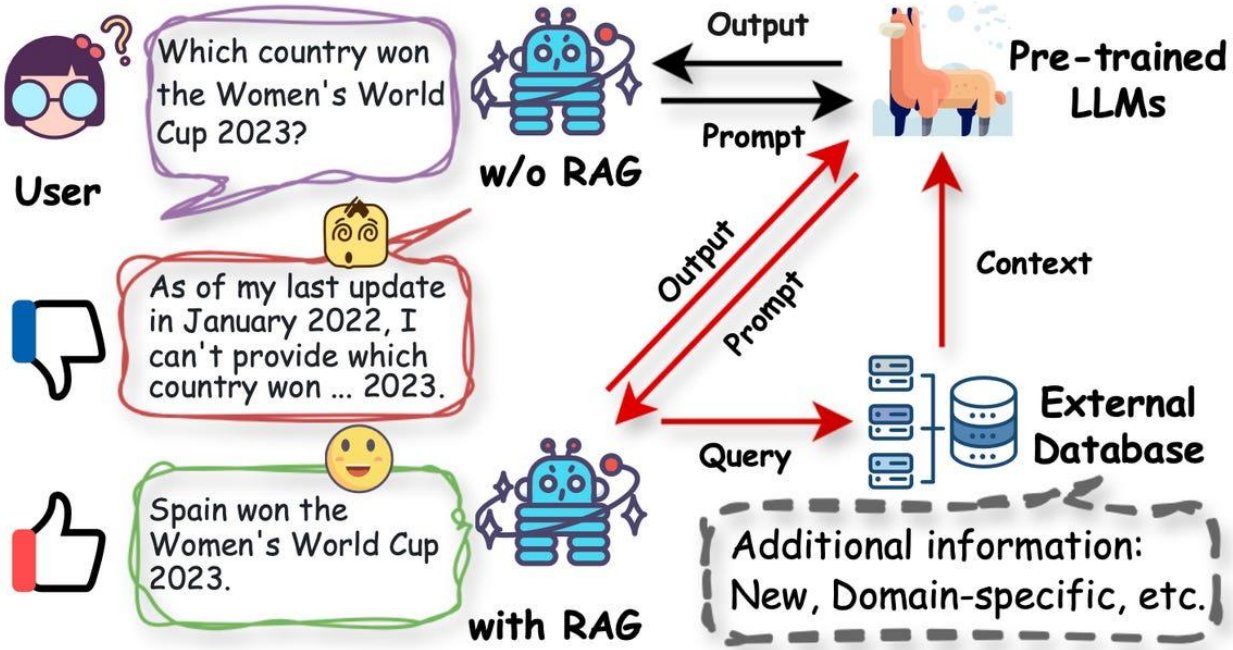
# Module Learning Objectives

- Understand the concept of Retrieval-Augmented Generation (RAG) and its components, including retrieval, generation, and augmentation
- Differentiate between dense and sparse retrieval processes and apply similarity measures like TF-IDF and cosine similarity
- Explore vector embeddings such as Word2Vec, GloVe, and BERT to enhance retrieval accuracy and contextual understanding
- Build and optimize RAG pipelines using modular architectures, dynamic configurations, and iterative retrieval techniques
- Measure the performance of RAG models using metrics like retrieval relevance and probabilistic scoring methods

# Retrieval-Augmented Generation

- New data and information are generated constantly; every second
- Not practical to retain LLMs such as BERT, T5-large, GPT-3, etc. continuously
  - 110M, 738M, and 175B parameters, respectively
- LLMs can sometimes provide inconsistent responses, as they may return random facts from their training data that are not related to the user's query
- **Solution** is to combine information retrieved from both the trained model and external databases at the time of the query
- Retrieval-augmented generation (RAG), leverages the strengths of both **retrieval** and **generation**-based approaches
- May reduce hallucination, use dynamic knowledge, integrate domain-specific data
- Addresses LLM limitations in factual accuracy, traceability, and current information

# RAG Block Diagram




Source: ZDNet. Make room for RAG: How Gen AI's balance of power is shifting. Accessed Nov 2024.

# Evolution of RAG

- Naive RAG
  - Simple indexing, retrieval, and generation chain
  - Challenges: Hallucination, redundancy, and context integration
- Advanced RAG
  - Optimized pre- and post-retrieval processes
  - Techniques: Sliding windows, query rewriting, and re-ranking
- Modular RAG
  - Flexible modules (e.g., search, memory, routing)
  - Enables end-to-end training and dynamic pipeline reconfiguration

# RAG Framework

- No rules on whether RAG models access an external or internal database first
- Design choices and steps taken
  - The initial input/query from the user
  - The LLM retrieves relevant information from external data sources
  - Framework combines **external** information with the LLM **internal** model
  - the LLM generates a coherent and contextually relevant response
- **Retrieval** text, structured data, knowledge graphs, LLM-generated content
- **Generation** by LLM fine-tuned for specific tasks, e.g. context curation
- **Augmentation** of iterative, recursive, or adaptive retrieval to refine responses
  - Incorporates multi-step reasoning and real-time adjustments

# Evaluation Metrics

- **Retrieval relevancy** to a query based on term importance, e.g. TF-IDF
- **TF-IDF** (Term Frequency-Inverse Document Frequency)
- Evaluates how important a word is to a document in a corpus
- Term frequency $\mathrm{TF}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$ of a term $t$ in document $d$
- Inverse document frequency $\mathrm{IDF}(t,D) = \log \frac{N}{1+|\{d \in D : t \in d\}|}$
  - $N$ is the size of the document corpus $D$
- TF-IDF score is $\mathrm{TF{-}IDF}(t,d,D) = \mathrm{TF}(t,d) \times \mathrm{IDF}(t,D)$
  - IDF normalizes frequent words such as stop words (e.g., 'the')
- Incorrect **assumption** is that terms (dimensions) are considered to be independent

# Vector Embeddings

- Compute **similarity** between queries and document vectors, semantic retrieval
- Embeddings are **dense** vector representations of text
- Unlike **sparse** vectors like TF-IDF, embeddings capture semantic relationships
- **Word2Vec** uses a shallow neural network to learn word embeddings
  - **Skip-gram** predicts the context words $w_{\text{context}}$ given a center word $w_{\text{center}}$
  - $\max \prod_{t=1}^{T} \prod_{-c \le j \le c, j \ne 0} \mathrm{P}(w_{t+j} \mid w_t)$, and
  - $\mathrm{P}(w_{\text{context}} | w_{\text{center}}) = \frac{\exp(v_{\text{context}}^{\top} v_{\text{center}})}{\sum_w \exp(v_w^{\top} v_{\text{center}})}$, $v$ is the word embedding vector

# GloVe and BERT

- **GloVe** (Global Vectors)
  - Embeddings are learned based on co-occurrence statistics of words in a corpus
  - $J = \sum_{i,j} f(X_{ij}) \left(v_i^\top v_j + b_i + b_j - \log X_{ij}\right)^2$ where $X_{ij}$ is the co-occurrence matrix
- **BERT** (Bidirectional Encoder Representations from Transformers)
  - Generates context-aware embeddings using a Transformer architecture
  - Given input tokens $X$

    $H^{(l)} = \mathrm{Transformer}(H^{(l-1)})$
  - where $H$ is the hidden state and $l$ is the layer number, $H_i^{(l)}$ is the vector representation of $i$-th token at the $l$-th layer
  - Output embeddings depend on all tokens in the sequence

# Similarity Measures

- Similarity measures determine how closely a retrieved document aligns with a query
- **Cosine Similarity** measures the cosine of the angle between two vectors
  - $\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\|\|B\|}$
  - Values range from -1 to 1, where 1 indicates perfect similarity
- **Euclidean Distance** calculates the straight-line distance between two vectors
  - $\text{Euclidean Distance}(A, B) = \sqrt{\sum_{i=1}^{n}(A_i - B_i)^2}$
  - As the number of dimensions ($n$) increases, distances between points tend to become similar
    - The contribution of each dimension diminishes in high-dimensional spaces
  - Thus, Cosine Similarity is preferred in NLP

# Example Sentence Tokenizer

```
from nltk.tokenize import word_tokenize
from sklearn.preprocessing import MultiLabelBinarizer

tokens1, tokens2 = word_tokenize(Text1), word_tokenize(Text2)
mlb = MultiLabelBinarizer()
X = mlb.fit_transform([tokens1, tokens2])

# X[0], X[1] are sentence vectors, plain tokenized
# such as
# array([[0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1,
#         0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1],
#        [1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0,
#         1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1]])
#
```

# Example Implementation

```
def sim_costeta(_a:np.ndarray, _b:np.ndarray)->float:
    from numpy.linalg import norm
    assert _a.shape == _b.shape, 'require vectors same shape'
    return np.dot(_a, _b) / (norm(_a)*norm(_b))

def sim_jaccard(_a:np.ndarray, _b:np.ndarray)->float:
    ix = np.where((_a>0.) & (_b>0.))
    ixd = np.where((_a>0.) | (_b>0.))
    return ix[0].shape[0] / ixd[0].shape[0]
```

# Probabilistic Models

- Probabilistic methods model the **uncertainty** of retrieved document relevance
- **Bayesian Inference** given prior $\mathrm{P}(H)$ and likelihood $\mathrm{P}(E|H)$
  - $\mathrm{P}(H|E) = \frac{\mathrm{P}(E|H)\mathrm{P}(H)}{\mathrm{P}(E)}$
  - $\mathrm{P}(H|E)$ is the posterior probability of hypothesis $H$ given evidence $E$
- Measures the quality of retrieved text chunks before they are fed into the LLM
- Calculate the posterior probability of each chunk being relevant to the query
  - **Likelihood** probability of the chunk providing a quality answer given its content
  - **Prior** the inherent likelihood of the chunk's relevance
    - can be influenced by its position within the source document

# Optimization

- End-to-end optimization is challenging
- The retrieval process is non-differentiable
  - Stochastic expected utility maximization is introduced
  - A utility function measures the similarity between the generated text and the ground truth, using metrics like BLEU or ROUGE, and RAG
  - **Expected Utility** is calculated as the average utility across the training set
  - Output space is often large/unlimited so it is approximated by sampling
- The probability of generating an output is modeled by $\sum \mathrm{P}(\text{selected docs})$
  - Exact computation is impractical due to the large document pool
  - An approximation is made using Gumbel-Top-k sampling, which introduces Gumbel noise to document scores and selects the top k documents
  - Output space is updated by **beam search**, including ground truth outputs

# Retrieval-Augmented LLMs

- Retrieval-Augmented Large Language Models (RA-LLMs)
  - Find helpful information, retrieval
  - Create responses, generation
  - Improve the answers, augmentation
- Retrieval process
  - Sparse and dense retrieval models
- Generation process of creating responses based on retrieved information
  - White-box and black-box models
- Augmentation integrates information from internal and external resources
  - Input-layer, output-layer, and intermediate-layer integrations

# Sparse Retrieval Process

- Typically relies on keyword matching and vector space models, TF-IDF or BM25
- Example user's query: "Can you tell me about red fruits?"
- Collection of corpus
  - Doc1: Apples are a type of fruit. They could be red, green, or yellow.
  - Doc2: Bananas are a good source of potassium. They are yellow and sweet.
  - Doc3: Oranges are citrus fruits that are rich in vitamin C.
- Use TF-IDF to obtain scores to get a relevance score for each doc wrt to the query
  - Doc1: High score because it contains "red" and "fruit"
  - Doc2: Low score because it does not contain relevant terms
  - Doc3: Medium score because it contains "fruit" but no "red"
- Result: The sparse retrieval process would return Doc1 as the most relevant

# Dense Retrieval Process

- Dense is based on embedding vectors rather than keyword search
- Use embeddings generated by NN to represent docs and queries
  - Vector similarity search (e.g., cosine similarity) used to find relevant information
  - Dense retrieval also often uses models like BERT to generate these embeddings
- User's query: "Best Italian restaurants near me"
  - A collection of restaurant descriptions exists, represented as dense vectors
  - e.g., "A cozy Italian eatery serving homemade pasta in a warm atmosphere."
- Identify the most relevant restaurant
  - Compute the similarity between the query vector and the restaurant vectors
  - Retrieves the restaurant whose vector is the closest to the query vector

# White-box Generation Process

- Allow for adjustments to their internal settings, providing flexibility in their use
- Encoder-Decoder models, such as T5 and BART
  - Process input and response separately; connect them with attention mechanism
- Decoder-only models
  - Self-attention layer
  - Only the decoder stack for autoregressive text generation
  - Handle both input and output simultaneously
- White-box models can help developers fine-tune how the model manages input

# Augmentation Input-Layer Integration

- Combining the query and retrieved docs to the generation model
- In-Context RALM concatenates the input and retrieved docs into a single sequence
- FID processes each retrieved document independently in the encoder
- Atlas and REPLUG concatenate the query with one retrieved document at a time
- Input-layer integration is commonly used in black-box generation-based RAG systems
- **Prompt Retrieval** involves LLMs using retrieved content as additional prompts
  - Helps improve LLMs using context without requiring detailed prompt engineering
  - Enhance the model's **zero-shot learning** capabilities
- References: FID Hofstätter 2023, Atlas Izacard 2023, REPLUG Shi 2023.

# Augmentation Output-Layer Integration

- Combine retrieval and generation results by the model refining its output
- kNN-LM blends predicted outcomes of the LM and the nearest neighbors found in the retrieval corpus
  - A simple method hinders the model from fully reasoning with the retrieved text
- Adaptive semiparametric LM added a gating network to post-process retrieved data
- REFEED uses an LLM to evaluate the retrieved info and refine the generated answer
- COMBO framework matches generated text with retrieved docs and uses a Fusion-in-Decoder-based approach to produce a final, enhanced answer
- References: Adaptive semiparametric Yogatama 2021, REFEED Yu 2023, COMBO Zhang 2023.

# Intermediate-Layer Integration

- Sophisticated approach by incorporating retrieved info into the model's internal layers
- Uses a semi-parametric module, adds complexity but can significantly enhance the model's performance through effective training
- A Transformer integrates the retrieved data encoded as dense representations
- RETRO (Retrieval-Enhanced Transformer) introduces a Chunked Cross Attention (CCA) layer to process retrieved chunks during the generation process
- kNNAugmented Attention Layer is proposed as an improvement
- EAE and TOME, utilize memory layers to incorporate retrieved entities and mentions
- Intermediate-layer integration requires access to the internals of generation models
  - Not possible with most LLMs available through inference APIs
- References: RETRO Borgeaud 2022, kNNAugmented Attention Wu 2022, EAE Févry 2020, TOME Jong 2022.

# RA-LLMs Training

- Fine-tuning LLMs to keep their knowledge up-to-date is challenging
- Retriever mechanisms help LLM utilize external knowledge dynamically
  - Avoiding full retrain
- **Training-free** methods for integrating retrieved knowledge into LLMs include
  - Prompt Engineering-based, such as In-Context RALM, IRCoT, GENREAD, and SKR
  - Retrieval-Guided Token Generation Methods, such as KNN-LMs and Rest which employ a non-parametric retrieval datastore
- References: In-Context RALM Ram 2023, IRCoT Trivedi 2023, GENREAD Yu 2023, SKR Wang 2023, KNN-LMs He 2021, Rest He 2023.

# RA-LLMs Training

- **Sequential Training** improves the synergy between a retriever and a LM
- The training coordinates retriever and generator efforts
- **Retriever First** has the retriever pre-trained, fixed while the LLM is next trained
  - RETRO uses a pre-trained BERT retriever, and an encoder-decoder to integrate
  - ITER-RTGEN uses S-BERT as the retriever and T5 as the generator
  - SMALLCAP has a multimodal CLIP and a GPT-2 as a decoder to produce captions
- **LLMs First** has the LLM pre-trained and then used to supervise retriever training
  - DKRR leverages LLM attention scores to guide retriever training
  - UPRISE has a frozen LLM to guide a prompt retriever, task-specific retrieval during inference
- References: SMALLCAP Ramos 2023, DKRR Izacard 2021, UPRISE Cheng 2023.

# RA-LLMs Training

- **Independent Training** the retriever and LLM are trained separately, no interaction
- LLM is trained by negative log-likelihood loss to generate outputs for given inputs
- Retrievers are categorized as either sparse or dense
- Sparse retrievers use sparse features, such as TF-IDF and BM25
- Dense retrievers use dense encoded queries and documents into dense vectors
- Some examples are DPR and CoG
- **Joint Training** optimizes retriever and generator together in an end-to-end manner
- REALM uses a Maximum Inner Product Search (MIPS) to locate the most relevant docs and jointly minimizes the negative log-likelihood to train both components
- References: DPR Karpukhin 2020, CoG Lan 2022, REALM Guu 2020.

# Data Conflict Resolution

- LLMs can behave differently when there is a conflict between training data and the information retrieved from external sources
- LLM can handle the situation by presenting both perspectives to the user
- Or it can synthesize the information to resolve the conflict
- The model can use the context of the query to determine which source of information is more relevant
  - Example, if the query specifically pertains to recent events or updates, the model is more likely to rely on external data sources
- Some systems may incorporate user feedback to refine their responses
- If users consistently indicate that certain information is outdated or incorrect, the model can adjust its reliance on different data sources accordingly
- Prompt engineering can help. How?

# Ethical Challenges

- Challenges apply as usual to Large Language models, RAG adds more risks
- **Bias and Fairness**
  - Ensuring that the retrieved information and generated responses are free from bias and fair is a significant challenge
- **Data Privacy and Security**
  - Accessing and retrieving data from external sources raises concerns about data privacy and security

# Fun Fact

- At the present, ChatGPT itself does not use RAG in its core functionality
- Instead, it relies on a large language model trained on a diverse dataset to generate responses based on the input it receives

- Noted as of Nov 2024

# Future of RAG

- Advanced retrieval techniques, especially via transformers, semantic search
- Multimodal RAG models
- Domain-specific fine-tuning, such as healthcare or finance
- Real-time adaptation and learning
- Enhanced contextual understanding, a common problem of LLMs/prompts
- Bias mitigation and fairness, via techniques of bias detection, fairness-aware training
- Scalable and efficient architectures, deployed in various envs, low-resource settings
- Robust data privacy and security measures, federated learning
- Integration with human-in-the-loop systems
- Ethical and responsible AI practices, ethical practices, transparency, and fairness

# References


- Foster, David. Generative deep learning. " O'Reilly Media, Inc.", 2023.
- Gao, Yunfan, et al. "Retrieval-augmented generation for large language models: A survey." arXiv preprint arXiv:2312.10997 (2023).
- Fan, Wenqi, et al. "A survey on rag meeting llms: Towards retrieval-augmented large language models." Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2024.

JOHNS HOPKINS

WHITING SCHOOL
*of* ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.